SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM111
Name: Parshwa Ramesh Bhavare
Division: B
Date: 11 October 2026
GitHub / Portfolio Link (optional): https://github.com/ParshwaBhavare?tab=repositories

PART A: Architecture Decision Record (ADR)

Title: One shared adjacency-list Graph Store with a visited array, and a separate Profiler / Timer module, for a fair BFS vs DFS comparison
Status: Accepted
Date: 11 October 2026

1. Context

In SLE-2 I had to compare BFS and DFS on three deterministic graphs: a 10-vertex tree (9 edges), K10 (45 edges) and K20 (190 edges). The comparison is only meaningful if both algorithms receive exactly the same input and are timed in exactly the same way. A single Python traversal of such a small graph is too short to time reliably, and timings change from run to run. K10 and K20 also contain many cycles, so a traversal must not revisit vertices. In SLE-3 I therefore had to decide how to store the graph and how to structure the system so that fairness is built into the design.

2. Decision

Every test graph is stored once as an adjacency list in a single Graph Store, and both bfs() and dfs() read it from start vertex 0, using a visited array so each vertex is processed once. Timing is done by a separate Profiler / Timer container (time.perf_counter, 3 runs of 10,000 traversals each, averaged), kept outside the Traversal Engine.

3. Alternatives Considered

- **Alternative 1:** Adjacency matrix. Rejected: every traversal scans V^2 cells, which wastes work on the sparse tree and uses more memory.
- **Alternative 2:** Separate graph structures per algorithm, with timing code written inside bfs() and dfs(). Rejected: the two algorithms would not be guaranteed the same input and timing method, and timing code would mix with search logic.
- **Alternative 3:** Time one traversal only (no repetition, no visited array shortcuts). Rejected: one traversal is too short to measure and cannot show run-to-run variation; without a visited check, K10/K20 would loop.

4. Rationale / Justification

Efficiency (SLE-2 analysis): both algorithms scan each vertex and each adjacency entry once, so $T = \Theta(V + E)$ with an adjacency list.

> **Cost of one traversal (undirected graph):** **Adjacency list = $\Theta(V + 2E)$ | Adjacency matrix = $\Theta(V^2)$**
> Tree: 10 + 18 = 28 steps vs 100 cells. K20: 20 + 380 = 400 steps vs 400 cells (list never loses).

Evidence from SLE-2 (same graph object, same start vertex, 3 runs × 10,000 traversals):

Test case	BFS avg (ms)	DFS avg (ms)	DFS vs BFS	Graph (adjacency entries)
Best (tree)	11.888	15.827	≈ 33% slower	V=10, E=9 (18 entries)
Average (K10)	24.991	26.793	≈ 7% slower	V=10, E=45 (90 entries)
Worst (K20)	73.183	67.249	≈ 8% faster	V=20, E=190 (380 entries)

The winner flips with density: BFS is faster on the sparse tree and K10, while DFS is faster on K20. These differences are only 7–33%, so they are meaningful only because the input and timing method were identical for both. Run-to-run noise is also real: DFS on the Average case ranged from 24.246 to 31.642 ms across its three runs, which is why I average three batches instead of trusting one. Repeating each traversal 10,000 times keeps every batch above 1 ms, so it is measurable. From SLE-3, keeping the Profiler outside the Traversal Engine means timing code never touches algorithm code, and the shared Graph Store and Visited Set keep the design of both searches identical, so any difference comes from the queue-vs-recursion strategy alone.

5. Consequences

Positive:

- Fair, reproducible comparison: same data, start vertex, visited logic and timer for BFS and DFS.
- Modular: a new algorithm (e.g. A*) or a new graph type can be added without changing the other containers.

Negative / Trade-offs:

- More modules and files than a single script would need for a small project.
- Timings are machine-dependent, and 10,000 back-to-back repetitions measure warm, repeated runs, not a single cold run.
- Recursive DFS is limited by Python's recursion depth on very large graphs, and only three graph sizes were tested, so scaling claims rest on the Θ(V + E) analysis.

6. References (optional)

GitHub: https://github.com/ParshwaBhavare?tab=repositories | SLE-2 Profiling Report (summary table, Fig. 2) | SLE-3 Container Diagram (Fig. 2) and Design Decisions section.

PART B: Short Summary of My SLE Journey

SLE-1: I built BFS and DFS search code on adjacency-list graphs, with an AI contribution log and README. SLE-2: I profiled both algorithms on a 10-vertex tree, K10 and K20 using 3 runs of 10,000 traversals, and found that both are Θ(V + E), with BFS faster on the sparse and medium graphs (11.888 vs 15.827 ms and 24.991 vs 26.793 ms) and DFS faster on K20 (67.249 vs 73.183 ms). SLE-3: I drew the C4 model (Context, Container, Component, Code) and separated the Graph Store, Traversal Engine, Visited Set, Profiler and Output modules. SLE-4: this ADR records the key decision connecting them, that all algorithms share one Graph Store and are timed by an independent Profiler, because it is what makes the SLE-2 numbers trustworthy.

PART C: AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** drafting this ADR in the required structure, organising my SLE-2 numbers into the Rationale, and suggesting viva questions.
- **What I did myself:** chose the decision from my own SLE-2 and SLE-3 work, checked every number against my SLE-2 tables and every module name against my SLE-3 diagrams, edited the final text, and prepared my own viva answers.

PART D: Viva Preparation Notes

- **Why this design?** Same graph + same timer = fair comparison; Profiler is separate from the Traversal Engine.
- **Profiling result:** BFS faster on tree (11.888 vs 15.827 ms) and K10; DFS faster on K20 (67.249 vs 73.183 ms). Timings are machine-dependent; the Θ(V + E) analysis is the main conclusion.
- **Why adjacency list, not matrix?** Tree: 28 steps vs 100 cells; K20 both about 400, so the list is never worse.
- **If the graph gets larger:** both grow as Θ(V + E); BFS queue and DFS stack are Θ(V); recursive DFS may hit Python's recursion limit, so I would switch to an iterative stack.

- **C4 Container in 1 minute:** Input → Graph Store → Traversal Engine (with Visited Set) ↔ Profiler / Timer → Output Module.